UNITÀ 2 · WEB SERVICE · LEZIONE 11

Testare le API con curl e Postman

Un'API funziona quando risponde bene anche alle richieste sbagliate. Oggi BiblioAPI finisce sotto esame: a mano, con una collezione di test e con uno script.

5G TPSI curl, Postman e Bruno, test automatici, casi negativi, OpenAPI

PER COMINCIARE

Come controlli di solito che un tuo programma funzioni?

SONDAGGIO

- **A** Lo eseguo una volta e guardo se va
- **B** Provo anche input sbagliati
- **C** Scrivo test automatici
- **D** Lo fa provare a un compagno
- **E** Se compila, funziona

PERCHÉ TESTARE UN'API

Quattro motivi, quattro strumenti

1

Esplorare

Capire come risponde davvero un servizio: curl.

2

Verificare

Controllare codici, header e corpo: test in Postman o Bruno.

3

Non regredire

Rieseguire tutto dopo ogni modifica: script e runner.

4

Documentare

Una collezione o un file OpenAPI dice come si usa l'API.

> Un test ha sempre tre parti: **la richiesta**, **il risultato atteso** scritto prima, **il confronto**. Senza il risultato atteso è solo una prova.

PUZZLE

Abbina ogni opzione di curl a ciò che fa

`-i`	---	Invia come corpo il contenuto di un file
`-v`	---	Aggiunge un header alla richiesta
`-X DELETE`	---	Mostra tutto lo scambio: richiesta inviata e risposta
`-H "Accept: …"`	---	Mostra anche riga di stato e header della risposta
`-d @libro.json`	---	Sceglie il metodo HTTP
`-w "%{http_code}"`	---	Stampa il codice di stato alla fine

LEGGERE CURL -V

Asterisco, maggiore, minore

```
* Connected to localhost (127.0.0.1) port 8000
> GET /api/libri/2 HTTP/1.1
> Host: localhost:8000
> User-Agent: curl/8.5.0
> Accept: */*
>
< HTTP/1.1 200 OK
< Date: Mon, 19 Oct 2026 15:22:22 GMT
< Content-type: application/json; charset=utf-8
< Content-length: 75
<
{"id":2,"titolo":"Il barone rampante",…}
```

- `*` informazioni di curl: DNS, connessione TCP, TLS.
- `>` righe **inviate**: la richiesta.
- `<` righe **ricevute**: la risposta.
- La riga vuota separa gli header dal corpo.
- `Content-length: 75` sono byte, non caratteri.

AL COMPUTER · 20 minuti

Un giro completo di BiblioAPI con curl

```
curl -i http://localhost:8000/api/libri
curl -i -X POST -H "Content-Type: application/json" -d @libro.json http://localhost:8000/api/libri
curl -i -X PUT -H "Content-Type: application/json" -d @libro2.json http://localhost:8000/api/libri/4
curl -v -X DELETE http://localhost:8000/api/libri/4
```

1. Riavvia BiblioServer: i dati tornano ai tre libri iniziali.
2. Prepara **libro.json** e **libro2.json** (stesso libro, anno diverso).
3. Esegui i comandi e salva l'output in **giro-curl.txt**, aggiungendo `>> giro-curl.txt 2>&1`.

Controllo: 200, poi **201** con `Location: /api/libri/4`, poi 200 con l'anno nuovo, poi `< HTTP/1.1 204 No Content` senza corpo.

QUIZ A TRABOCCHETTO

curl con `-d @libro.json` ma senza `-X`: che cosa parte verso BiblioAPI?

QUIZ

A Una GET con il file nella query string

B Un POST con Content-Type application/json

C Un POST con Content-Type application/x-www-form-urlencoded

D Un errore: senza -X curl non sa che metodo usare

OLTRE LA RIGA DI COMANDO

Client grafici per le API

Postman

Il più diffuso nelle aziende. Collezioni, ambienti, test in JavaScript, runner, documentazione.

Per salvare collezioni e ambienti serve un account.

Bruno

Open source, funziona offline e senza account. Le collezioni sono file di testo in una cartella: si mettono su Git.

Hoppscotch

Open source, si usa dal browser. Per chiamare `localhost` serve la sua estensione o l'app.

- **Collezione**: le richieste di un'API, in cartelle.
- **Ambiente**: variabili come `{{base}}` = `http://localhost:8000/api`, da cambiare per passare dal PC al server di produzione.

TEST AUTOMATICI

Stesso test, due strumenti

Postman · scheda Scripts, Post-response

```
pm.test("201 Created", () =>
  pm.response.to.have.status(201));
pm.test("ha un id numerico", () => {
  const libro = pm.response.json();
  pm.expect(libro.id).to.be.a("number");
  pm.collectionVariables.set("id", libro.id);
});
```

Bruno · scheda Tests

```
test("201 Created", () => {
  expect(res.getStatus()).to.equal(201);
});
test("ha un id numerico", () => {
  const libro = res.getBody();
  expect(libro.id).to.be.a("number");
  bru.setVar("id", libro.id);
});
```

Il test salva l'id del libro creato: la richiesta successiva usa `{{base}}/libri/{{id}}`.

AL COMPUTER · 35 minuti

La collezione di test di BiblioAPI

1. In Bruno (o Postman) crea la collezione **BiblioAPI** e un ambiente **locale** con `base` = `http://localhost:8000/api`.
2. Aggiungi in ordine: elenco, POST nuovo libro, GET `{{id}}`, PUT `{{id}}`, DELETE `{{id}}`, GET `{{id}}`.
3. Ogni richiesta ha almeno un test sul codice; il POST salva `id`.
4. Avvia il **Runner** sull'intera collezione, due volte di fila.
5. Esporta la collezione (in Bruno: la cartella in zip).

Controllo: 6 richieste, tutte verdi, anche alla seconda esecuzione: 200, 201, 200, 200, 204, **404**. Se la seconda volta qualcosa fallisce, un test dipende da un id scritto a mano.

PUZZLE

Metti in ordine la collezione: perché ogni test regga da solo

1. GET /libri/{{id}}: il libro esiste e ha il titolo inviato
2. DELETE /libri/{{id}}: 204
3. PUT /libri/{{id}} con l'anno cambiato
4. POST /libri e salva l'id ricevuto
5. GET /libri/{{id}}: l'anno è quello nuovo
6. GET /libri/{{id}}: 404

I CASI NEGATIVI

Le richieste sbagliate che un'API deve reggere

Richiesta	Che cosa simula	Atteso
`GET /api/libri/9999`	Un id che non esiste	**404**
`GET /api/libri/abc`	Un id che non è un numero	**400**
`POST /api/libri {"titolo":"X"}`	Campi obbligatori mancanti	**400**
`POST /api/libri {"titolo":`	JSON non valido	**400**
`PATCH /api/libri`	Un metodo non previsto	**405**
`GET /api/autori`	Una risorsa che non esiste	**404**

In ogni caso il server deve restare acceso e rispondere con un JSON d'errore: mai un 500, mai uno stack trace.

METTITI ALLA PROVA

Tutti i test della collezione sono verdi. Che cosa sappiamo davvero?

QUIZ

A Che l'API si comporta come atteso nei casi che abbiamo provato

B Che l'API non ha bug

C Che l'API è sicura

D Che l'API regge mille utenti insieme

MENTI CHE CI HANNO PRECEDUTO

Il limite di ogni test

> **Il test dei programmi può servire a mostrare la presenza di bug, ma mai a dimostrarne l'assenza!**

Edsger W. Dijkstra, informatico olandese, premio Turing 1972, in *Notes on Structured Programming*, 1970



AL COMPUTER · 20 minuti

Uno script che controlla tutto in un secondo

```
@echo off
rem test-api.bat: in un file .bat il % si scrive %%
set B=http://localhost:8000/api/libri
curl -s -o NUL -w "GET elenco      %%{http_code}\n" %B%
curl -s -o NUL -w "POST nuovo      %%{http_code}\n" ^
  -X POST -H "Content-Type: application/json" -d @libro.json %B%
curl -s -o NUL -w "GET 4           %%{http_code}\n" %B%/4
curl -s -o NUL -w "DELETE 4        %%{http_code}\n" -X DELETE %B%/4
curl -s -o NUL -w "GET 4 di nuovo  %%{http_code}\n" %B%/4
curl -s -o NUL -w "POST senza dati %%{http_code}\n" -X POST -d "{}" %B%
curl -s -o NUL -w "PATCH elenco    %%{http_code}\n" -X PATCH %B%
```

1. Salva **test-api.bat** accanto a libro.json.
2. Riavvia il server ed esegui lo script.
3. Aggiungi due casi negativi della tabella.

Controllo: a server appena avviato: 200, 201, 200, 204, 404, 400, 405, uno per riga.

IL CONTRATTO DELL'API

OpenAPI: la documentazione che le macchine sanno leggere

```yaml
openapi: 3.1.0
info: { title: BiblioAPI, version: "1.0" }
paths:
  /api/libri/{id}:
    get:
      summary: Legge un libro
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200": { description: Il libro }
        "404": { description: Libro non trovato }
```

Da un file OpenAPI si generano una pagina di documentazione interattiva, una collezione per Postman o Bruno e perfino il codice dei client.



SFIDA A SQUADRE

Il lucchetto del tester



- 1ª cifra: la prima cifra del codice per GET di un libro cancellato
- 2ª e 3ª cifra: le ultime due cifre del codice per PATCH su /api/libri
- 4ª cifra: quante richieste ha la catena del puzzle, prima della DELETE

`codice`

BIGLIETTO D'USCITA

Completa prima di uscire

L'opzione di curl che mostra anche la richiesta inviata è _____.

Con `-d` e senza `-X`, curl usa il metodo _____.

Un metodo non previsto sulla risorsa deve ricevere _____.

Il formato standard per descrivere un'API REST si chiama _____.

Il test può mostrare la presenza dei bug, mai la loro _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- curl: -i, -v, -X, -H, -d @file, -w
- Collezioni, ambienti, variabili come {{id}}
- Ogni test: richiesta, atteso, confronto
- I casi negativi contano quanto i positivi
- Script e runner per non regredire
- OpenAPI: il contratto leggibile da persone e macchine

> **All'esame**: in un progetto, una tabella di casi di test con richiesta, risultato atteso e motivo dimostra che hai pensato anche a ciò che può andare storto.

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso.